❖ INTRODUCTION TO GENERATION LANGUAGES AND PROCESSOR

LANGUAGE PROCESSOR:

- **Compiler**:

A compiler is a software program that translates the entire source code of a high-level programming language into an equivalent set of machine-level instructions or a lower-level intermediate code.

It performs a one-time translation of the entire program before execution.

The output of a compiler is typically an executable file or binary code that can be run without the need for the original source code.

Compilation can take some time, but the resulting executable code is generally faster because it's optimized during the compilation process.

- **Interpreter**:

An interpreter is a program that reads and executes high-level code line by line or statement by statement, without the need for a separate compilation step.

It translates and executes code in real-time, often providing immediate feedback to the programmer.

Interpreters are commonly used in scripting languages like Python and JavaScript.

Code execution may be slower than with a compiler because of the on-the-fly translation.

- **Assemblers**:

An assembler is a program that translates assembly language code into machine code.

Assembly language is a low-level human-readable representation of machine code, using mnemonics and symbolic names for instructions and memory locations.

Assemblers are used to develop software at the hardware level, often for embedded systems or system-specific tasks.

Assembly code is specific to the architecture it's written for, so it's not as portable as high-level languages.

In summary, compilers and assemblers produce machine code as their output, while interpreters execute code directly. Compilers are typically used for languages like C or C++, which are compiled before execution. Interpreters are used for languages like Python or Ruby, which are executed line-by-line. Assemblers are used for low-level programming when you need to write code specific to a particular hardware architecture.

Difference:-

Assembler	Compiler	Interpreter
assembler translates mnemonics to machine code.	Converts High Level Language to Low Level Language.	Converts the higher Level language to low level language
Most Efficient Executable	entire code and converts it in to the Executable Code and runs the code.	single line of code converts it to the binary language
	C, C++ are Compiler based Languages.	BASIC is the Interpreter based Language.
	Slight Difficult to Execute	*Ex-PHP, Perl, Java Script.*
		interpreters are sometimes used during the development of a program

➢ GENERATION AND TYPES OF LANGUAGES:

1. Low Level Languages

The low-level language is a programming language that provides no abstraction from the hardware, and it is represented in 0 or 1 forms, which are the machine instructions. The languages that come under this category are the Machine level language and Assembly language.

2. High Level Languages

The high-level language is a programming language that allows a programmer to write the programs which are independent of a particular type of computer. The high-level languages are considered as high-level because they are closer to human languages than machine-level languages.

When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem.

A compiler is required to translate a high-level language into a low-level language.

Advantages of a high-level language

- The high-level language is easy to read, write, and maintain as it is written in English like words.
- The high-level languages are designed to overcome the limitation of low-level language, i.e., portability. The high-level language is portable; i.e., these languages are machine-independent.

Differences between Low-Level language and High-Level language

The following are the differences between low-level language and high-level language:

Low-level language	High-level language
It is a machine-friendly language, i.e., the computer understands the machine language, which is represented in 0 or 1.	It is a user-friendly language as this language is written in simple English words, which can be easily understood by humans.
The low-level language takes more time to execute.	It executes at a faster pace.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler to convert the high-level language instructions into machine code.

The machine code cannot run on all machines, so it is not a portable language.	The high-level code can run all the platforms, so it is a portable language.
It is memory efficient.	It is less memory efficient.

3. Machine-level language

The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1. As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so the instructions given to the computer can be only in binary codes. Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions.

It is error-prone as it is not easy to understand, and its maintenance is also very high. A machine-level language is not portable as each computer has its machine instructions, so if we write a program in one computer will no longer be valid in another computer.

The different processor architectures use different machine codes, for example, a PowerPC processor contains RISC architecture, which requires different code than intel x86 processor, which has a CISC architecture.

4. Assembly Language

The assembly language contains some human-readable commands such as add, sub, etc. The problems which we were facing in machine-level language are reduced to some extent by using an extended form of machine-level language known as assembly language. Since assembly language instructions are written in English words like add, sub, so it is easier to write and understand.

As we know that computers can only understand the machine-level instructions, so we require a translator that converts the assembly code into machine code. The translator used for translating the code is known as an assembler.

The assembly language code is not portable because the data is stored in computer registers, and the computer has to know the different sets of registers.

The assembly code is not faster than machine code because the assembly language comes above the machine language in the hierarchy, so it means that

assembly language has some abstraction from the hardware while machine language has zero abstraction.

- **Differences between Machine-Level language and Assembly language**

  The following are the differences between machine-level language and assembly language:

Machine-level language	Assembly language
The machine-level language comes at the lowest level in the hierarchy, so it has zero abstraction level from the hardware.	The assembly language comes above the machine language means that it has less abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the computer.	In assembly language, the assembler is used to convert the assembly code into machine code.
It is a first-generation programming language.	It is a second-generation programming language.

5. Next-Generation Languages:

"Next-generation languages" is not a standardized or widely recognized term. It could refer to several possibilities:

It might refer to languages that incorporate the latest advancements in technology, such as those designed for quantum computing.

It could also be used informally to describe languages that are more recent and may have advanced features or paradigms compared to older languages.

The field of programming languages is constantly evolving, so newer languages are developed to address specific needs and challenges.

➢ INTRODUCTION TO WEB TECHNOLOGY: SCRIPTING LANGUAGES

What is a Scripting Language?

- A scripting language is a type of programming language that is designed to be interpreted rather than compiled. Unlike compiled languages like C++ or Java, where code is transformed into machine code before execution, scripts are executed by an interpreter line by line.

- This makes scripting languages more versatile and allows developers to write and modify code rapidly without the need for a lengthy compilation process.

▪ Role of Scripting Languages in Web Development:

Scripting languages are essential in web development for several reasons:

1. Dynamic Content: Scripting languages enable developers to generate dynamic content on web pages. This means that web pages can respond to user interactions, display data from databases, and adapt to various conditions in real-time.

2. Client-Side Scripting: Scripting languages are used for client-side scripting, which means running code in a user's web browser. This enables interactivity and responsiveness, allowing users to interact with web applications without having to reload the entire page.

3.Server-Side Scripting: On the server-side, scripting languages are used to process requests, access databases, and generate dynamic web pages before sending them to the client. This is crucial for building complex web applications and handling user data securely.

4. Web Frameworks: Many web development frameworks and libraries are built on top of scripting languages. These frameworks simplify common web development tasks, such as handling HTTP requests, routing, and database interactions.

- ## Common Scripting Languages in Web Development:

Several scripting languages are commonly used in web development, Including:

1. JavaScript: JavaScript is the primary client-side scripting language for web development. It enables interactive and dynamic web content, and it can be used in conjunction with HTML and CSS to create modern web applications.

2. Python: Python is a versatile scripting language that is often used for server-side scripting in web development. It's known for its simplicity and readability, making it a popular choice for building web applications.

3. PHP: PHP is a server-side scripting language specifically designed for web development. It is used for tasks like processing forms, interacting with databases, and generating web content.

4. Ruby: Ruby is another scripting language used in web development, with the Ruby on Rails framework being a popular choice for building web applications.

5. Node.js (JavaScript on the server): Node.js allows developers to use JavaScript on the server-side, making it possible to write both client-side and server-side code using the same language.

- ## Advantages of scripting languages:

Easy learning: The user can learn to code in scripting languages quickly, not much knowledge of web technology is required.

Fast editing: It is highly efficient with the limited number of data structures and variables to use.

Interactivity: It helps in adding visualization interfaces and combinations in web pages. Modern web pages demand the use of scripting languages. To create enhanced web pages, fascinated visual description which includes background and foreground colors and so on.

Functionality: There are different libraries which are part of different scripting languages. They help in creating new applications in web browsers and are different from normal programming languages.